

# Comparative Data Structures and Algorithms

Implementations, Complexity, and Learning Evidence

T. R. Bentley

Repository: Tybent18/data-structures-practice  
Document class: repository-grounded technical report  
Edition: September 2026

## Scope

Foundational structures and algorithms implemented primarily in Python and C.

## Evidence statement

Claims in this report are limited to repository artifacts identified by filename. Proposed tests, benchmarks, integrations, and research directions are labeled as future work; they are not reported results.

# Abstract

This report organizes a multi-language practice collection around two evidence classes: data representation and algorithmic transformation. Linked lists, stacks, trees, structures, searching, sorting, and recursion are mapped to their source artifacts. Complexity expectations are stated as analytical baselines rather than benchmark results, and a measurement plan is defined for future empirical comparison.

## 1. Repository evidence map

| Cluster                 | Artifacts                             | Interpretation                                               |
|-------------------------|---------------------------------------|--------------------------------------------------------------|
| Linear structures       | linked_list.c, stack.c                | Pointer-linked storage and last-in-first-out behavior.       |
| Hierarchical structures | binary_tree.c                         | Tree representation and traversal foundations.               |
| Representation          | struct.c, user_def_struct.py          | Contrasts explicit C records with Python-defined structures. |
| Search                  | search.py, expand_search.py           | Search strategies and extension exercises.                   |
| Sort and recursion      | sort.py, expand_sort.py, recursion.py | Ordering transformations and recursive decomposition.        |

## 2. Technical framing

- How do language choices change implementation detail while preserving abstract behavior?
- Which operations dominate time and auxiliary-space costs?
- How can correctness be separated from performance measurement?

## 3. Reproducible inspection protocol

A reviewer should clone the repository, record the commit SHA and toolchain versions, inspect each mapped artifact, execute only examples whose entry points and prerequisites are declared, preserve standard output and errors, and compare observations with documented expectations. Failures in deliberate debugging targets are evidence only when the expected failure class is stated in advance.

## 4. Evidence maturity

| Level | Meaning            | Current interpretation                           |
|-------|--------------------|--------------------------------------------------|
| E0    | Artifact listed    | Source file exists in the repository             |
| E1    | Behavior described | README or report states intended behavior        |
| E2    | Execution recorded | Command, environment, and output are retained    |
| E3    | Behavior tested    | Repeatable assertions evaluate expected behavior |
| E4    | Comparative result | Frozen data supports a bounded comparison        |

## 5. Limitations

- Complexity statements are theoretical expectations, not measured timings.
- Implementations do not yet share a unified test corpus.
- The collection is educational and not optimized as a production library.

## 6. Development roadmap

- Add parameterized correctness tests.
- Create a frozen benchmark dataset across input sizes.
- Record operation counts and wall-clock distributions separately.

## 7. Portfolio role

This repository belongs to a cumulative foundation sequence: Foundations & Algorithms → Data Structures Practice → OOP Concepts → Math for Computing → Practical Utilities. The sequence demonstrates increasing abstraction and application; advanced repositories carry the stronger systems and empirical-research claims.

## Conclusion

The repository is most credible when its modest scale is treated as a strength: each artifact can be inspected directly, every claim can point to code, and future measurements can be added without rewriting history. Sophistication comes from traceability, not inflated labels.